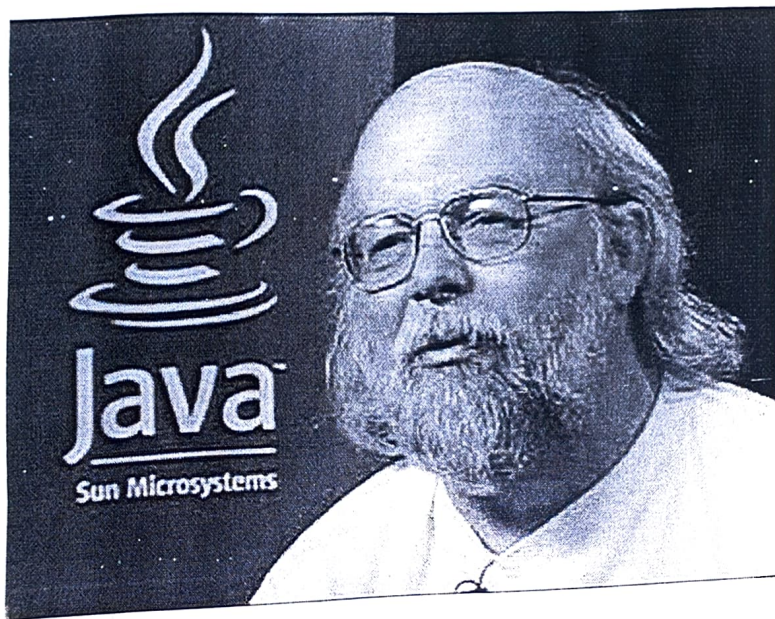


JAVA PROGRAMMING

Java – James Gosling



James Gosling is a famous Canadian software developer who has been with Sun Microsystems since 1984 and is considered as father of Java programming language. Gosling did the original design of Java and implemented its original compiler and virtual machine.

UNIT-I INTRODUCTION

History of Java:

In 1990, Sun Micro Systems Inc (US) was conceived a project to develop software for consumer electronic devices that could be controlled by a remote. This project was called Stealth Project but later its name was changed to Green Project.

In January 1991, Project Manager James Gosling and his team members Patrick Naughton, Mike Sheridan, Chris Wrath, and Ed Frank met to discuss about this project.

Gosling thought C and C++ would be used to develop the project. But the problem he faced with them is that they were system dependent languages. The trouble with C and C++ (and most other languages) is that they are designed to be compiled for a specific target and could not be used on various processors, which the electronic devices might use.

James Gosling with his team started developing a new language, which was completely system independent. This language was initially called **OAK**. Since this name was registered by some other company, later it was changed to **Java**.

James Gosling and his team members were consuming a lot of coffee while developing this language. Good quality of coffee was supplied from a place called "Java Island". Hence they fixed the name of the language as Java. The symbol for Java language is cup and saucer.

Sun formally announced Java at Sun World conference in 1995. On January 23rd 1996, JDK10 version was released.

Features of Java (Java buzz words):

Simple: Learning and practicing java is easy because of resemblance with c and C++.

Object Oriented Programming Language: Unlike C++, Java is purely OOP.

Distributed: Java is designed for use on network; it has an extensive library which works in agreement with TCP/IP.

Secure: Java is designed for use on Internet. Java enables the construction of virus-free, tamper free systems.

Robust (Strong/ Powerful): Java programs will not crash because of its exception handling and its memory management features.

Interpreted: Java programs are compiled to generate the byte code. This byte code can be downloaded and interpreted by the interpreter. Class file will have byte code instructions and JVM which contains an interpreter will execute the byte code.

Portable: Java does not have implementation dependent aspects and it yields or gives same result on any machine.

Architectural Neutral Language: Java byte code is not machine dependent, it can run on any machine with any processor and with any OS.

High Performance: Along with interpreter there will be JIT (Just In Time) compiler which enhances the speed of execution.

Multithreaded: Executing different parts of program simultaneously is called multithreading. This is an essential feature to design server side programs.

Dynamic: We can develop programs in Java which dynamically change on Internet (eg: Applets).

Obtaining the Java Environment:

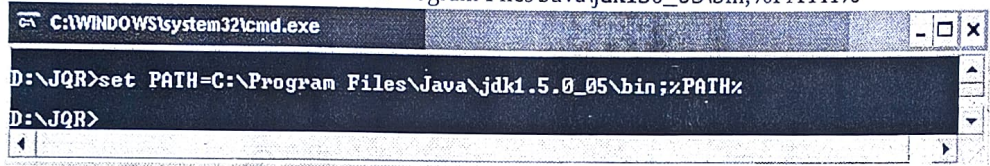
We can download the JDK (Java Development Kit) including the compiler and runtime engine from Sun at: <http://javasuncom/javase>

Install JDK after downloading, by default JDK will be installed in
C:\Program Files\Java\jdk150_05 (Here jdk150_05 is JDK's version)

Setting up Java Environment: After installing the JDK, we need to set at least one environment variable in order to be able to compile and run Java programs. A PATH environment variable enables the operating system to find the JDK executables when our working directory is not the JDK's binary directory.

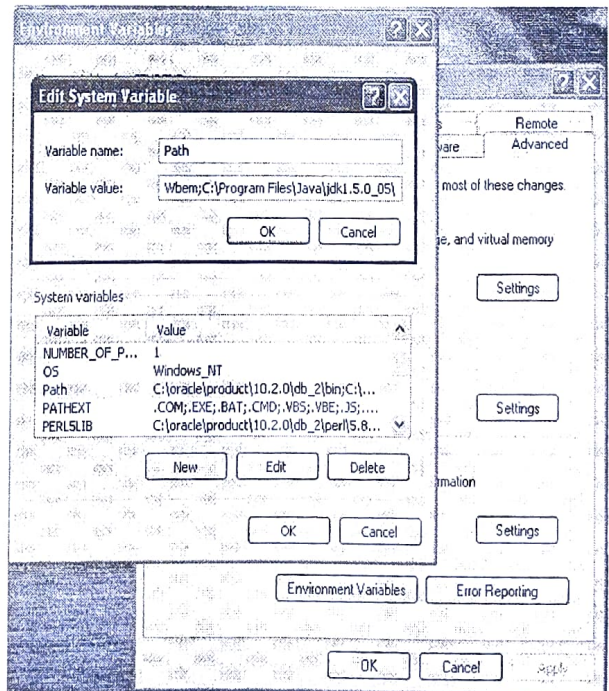
Setting environment variables from a command prompt: If we set the variables from a command prompt, they will only hold for that session. To set the PATH from a command prompt:

```
set PATH=C:\Program Files\Java\jdk150_05\bin;%PATH%
```



Setting environment variables as system variables: If we set the variables as system variables they will hold continuously.

- o Right-click on *My Computer*
- o Choose *Properties*
- o Select the *Advanced* tab
- o Click the *Environment Variables* button at the bottom
- o In system variables tab, select path (system variable) and click on edit button
- o A window with variable name-path and its value will be displayed
- o Don't disturb the default path value that is appearing and just append (add) to that path at the end:
;C:\ProgramFiles\Java\jdk150_05\bin;
- o Finally press OK button



Programming Structure

Comments: Comments are description about the aim and features of the program. Comments increase readability of a program. Three types of comments are there in Java:

Single line comments: These comments start with //

eg: // this is comment line

Multi line comments: These comments start with /* and end with */

eg: /* this is comment line*/

Java documentation comments: These comments start with /** and end with */

These comments are useful to create a HTML file called API (application programming Interface) document. This file contains description of all the features of software.

Structure of the Java Program:

As all other programming languages, Java also has a structure.

The first line of the C/C++ program contains include statement. For example, `<stdio.h>` is the header file that contains functions, like `printf()`, `scanf()` etc. So if we want to use any of these functions, we should include this header file in C/C++ program.

Similarly in Java, first we need to import the required packages. By default, `java.lang.*` is imported. Java has several such packages in its library. A package is a kind of directory that contains a group of related classes and interfaces. A class or interface contains methods. Since Java is purely an Object Oriented Programming language, we cannot write a Java program without having at least one class or object. So, it is mandatory to write a class in Java program. We should use the `class` keyword for this purpose and then write class name.

In C/C++, program starts executing from the `main` method. Similarly in Java, program starts executing from the `main` method. The return type of the `main` method is `void` because the program starts executing from the `main` method and it returns nothing.

Sample Program:

```
//A Simple Java Program
import java.lang.System;
import java.lang.String;
class Sample
{
    public static void main(String args[])
    {
        System.out.print ("Hello world");
    }
}
```

Since Java is purely an Object Oriented Programming language, without creating an object to a class it is not possible to access methods and members of a class. But the `main` method is also a method inside a class, since program execution starts from the `main` method we need to call the `main` method without creating an object.

Static methods are the methods, which can be called and executed without creating objects. Since we want to call the `main()` method without using an object, we should declare `main()`

method as static JVM calls main () method using its Classnamemain () at the time of running the program. JVM is a program written by Java Soft people (Java development team) and main () is the method written by us. Since, main () method should be available to the JVM, it should be declared as public. If we don't declare main () method as public, then it doesn't make itself available to JVM and JVM cannot execute it.

JVM always looks for main () method with String type array as parameter. Otherwise, JVM cannot recognize the main () method, so we must provide String type array as parameter to main () method.

A class code starts with a { and ends with a } A class or an object contains variables and methods (functions). We can create any number of variables and methods inside the class. This is our first program, so we had written only one method called main ()

Our aim of writing this program is just to display a string "Hello world". In Java, print () method is used to display something on the monitor.

A method should be called by using objectnamemethodname (). So, to call print () method, create an object to PrintStream class then call objectnameprint () method.

An alternative is given to create an object to PrintStream Class i.e. System.out. Here, System is the class name and out is a static variable in System class. out is called a field in System class. When we call this field a PrintStream class object will be created internally. So, we can

call print() method as: `System.out.print("Hello world");`

println () is also a method belonging to PrintStream class. It throws the cursor to the next line after displaying the result.

In the above Sample program System and String are the classes present in java.lang package.

Escape Sequence: Java supports all escape sequence which is supported by C/ C++. A character preceded by a backslash (\) is an escape sequence and has special meaning to the compiler. When an escape sequence is encountered in a print statement, the compiler interprets it accordingly.

Escape Sequence	Description
\t	Insert a tab in the text at this point
\b	Insert a backspace in the text at this point
\n	Insert a newline in the text at this point
\r	Insert a carriage return in the text at this point
\f	Insert a form feed in the text at this point
'	Insert a single quote character in the text at this point
"	Insert a double quote character in the text at this point
\\	Insert a backslash character in the text at this point

Creating a Source File:

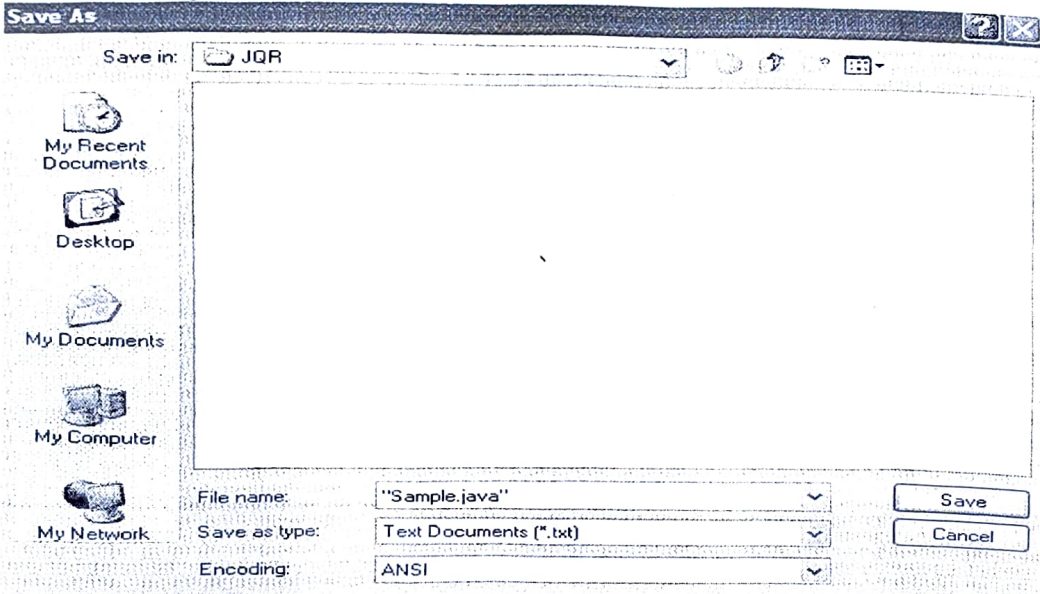
Type the program in a text editor (i.e. Notepad, WordPad, Microsoft Word or Edit Plus). We can launch the Notepad editor from the **Start** menu by selecting **Programs > Accessories > Notepad**. In a new document, type the above code (i.e. Sample Program).

Save the program with filename same as Class_name (i.e. Sample.java) in which main method is written. To do this in Notepad, first choose the **File > Save** menu item. Then, in

the Save dialog box:

Using the **Save in** combo box, specify the folder (directory) where you'll save your file

In the **File name** text field, type "Samplejava", including the quotation marks Then the dialog box should look like this:



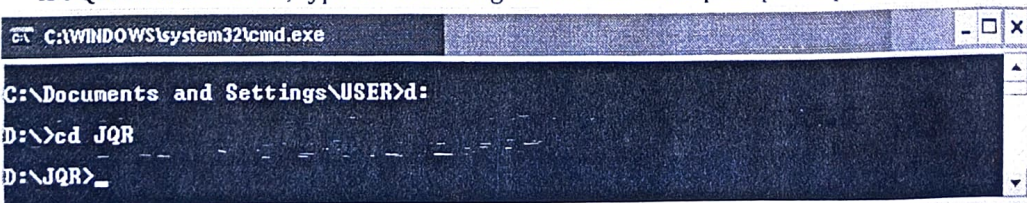
Now click **Save**, and exit Notepad

Compiling the Source File into a class File:

To Compile the Samplejava program go to DOS prompt We can do this from the **Start** menu by choosing **Run** and then entering cmd The window should look similar to the following figure



The prompt shows current directory To compile Samplejava source file, change current directory to the directory where Samplejava file is located For example, if source directory is JQR on the D drive, type the following commands at the prompt and press Enter:



Now the prompt should change to D:\JQR>

At the prompt, type the following command and press Enter
javac Samplejava


```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac Sample.java
D:\JQR>
```

The compiler generates byte code and Sampleclass will be created

Executing the Program (Sampleclass):

To run the program, enter java followed by the class name created at the time of compilation at the command prompt in the same directory as:

java Sample

```
C:\WINDOWS\system32\cmd.exe
D:\JQR>java Sample
Hello world
D:\JQR>
```

The program interpreted and the output is displayed

The Java Virtual Machine: Java Virtual Machine (JVM) is the heart of entire Java program execution process. First of all, the java program is converted into a class file consisting of byte code instructions by the java compiler at the time of compilation. Remember, this java compiler is outside the JVM. This class file is given to the JVM. Following figure shows the architecture of Java Virtual Machine.

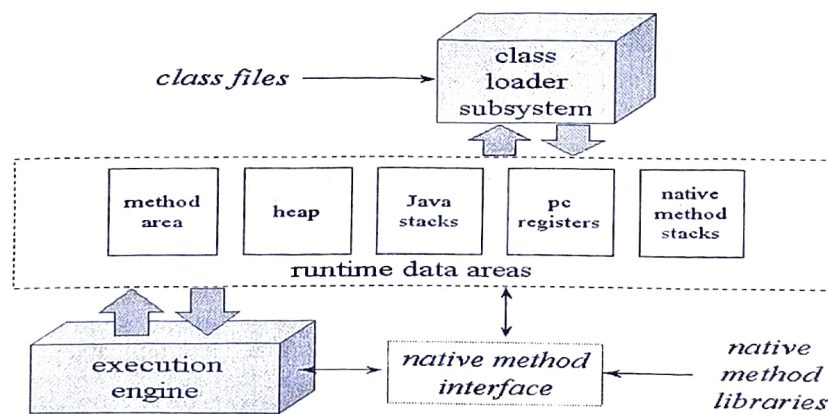


Figure: The internal architecture of the Java virtual machine

In JVM, there is a module (or program) called class loader sub system, which performs the following instructions:

First of all, it loads the class file into memory

Then it verifies whether all byte code instructions are proper or not. If it finds any instruction suspicious, the execution is rejected immediately.

If the byte instructions are proper, then it allocates necessary memory to execute the program. This memory is divided into 5 parts, called run time data areas, which contain the data and results while running the program. These areas are as follows:

- o **Method area:** Method area is the memory block, which stores the class code, code of the variables and code of the methods in the Java program (Method means functions written in a class)
- o **Heap:** This is the area where objects are created. Whenever JVM loads a class, method and heap areas are immediately created in it.
- o **Java Stacks:** Method code is stored on Method area. But while running a method, it needs some more memory to store the data and results. This memory is allotted on Java Stacks. So, Java Stacks are memory area where Java methods are executed. While executing methods, a separate frame will be created in the Java Stack, where the method is executed. JVM uses a separate thread (or process) to execute each method.
- o **PC (Program Counter) registers:** These are the registers (memory areas), which contain memory address of the instructions of the methods. If there are 3 methods, 3 PC registers will be used to track the instruction of the methods.
- o **Native Method Stacks:** Java methods are executed on Java Stacks. Similarly, native methods (for example C/C++ functions) are executed on Native method stacks. To execute the native methods, generally native method libraries (for example C/C++ header files) are required. These header files are located and connected to JVM by a program, called Native method interface.

Execution Engine contains interpreter and JIT compiler which translates the byte code instructions into machine language which are executed by the microprocessor. Hot spot (loops/iterations) is the area in class file i.e. executed by JIT compiler. JVM will identify the Hot spots in the class files and it will give it to JIT compiler where the normal instructions and statements of Java program are executed by the Java interpreter.

Naming Conventions, Data Types and Operators

Naming Conventions: Naming conventions specify the rules to be followed by a Java programmer while writing the names of packages, classes, methods etc.

Package names are written in small letters **eg:** javaio, javalang, javaawt etc.

Each word of class name and interface name starts with a capital **eg:** Sample, AddTwoNumbers

Method names start with small letters then each word starts with a capital **eg:** sum (), sumTwoNumbers (), minValue ()

Variable names also follow the same above method rule **eg:** sum, count, totalCount

Constants should be written using all capital letters **eg:** PI, COUNT

Keywords are reserved words and are written in small letters **eg:** int, short, float, public, void

Data Types: The classification of data item is called data type Java defines eight simple types of data byte, short, int, long, char, float, double and boolean These can be put in four groups:

Integer Data Types: These data types store integer numbers

Data Type	Memory size	Range
Byte	1 byte	-128 to 127
Short	2 bytes	-32768 to 32767
Int	4 bytes	-2147483648 to 2147483647
Long	8 bytes	-9223372036854775808 to 9223372036854775807

eg: byte rno = 10;
long x = 150L; L means forcing JVM to allot 8 bytes

Float Data Types: These data types handle floating point numbers

Data Type	Memory size	Range
Float	4 bytes	-34e38 to 34e38
Double	8 bytes	-17e308 to 17e308

eg: float pi = 3142f;
double distance = 198e8;

Character Data Type: This data type represents a single character char data type in java uses two bytes of memory also called Unicode system Unicode is a specification to include alphabets of all international languages into the character set of java

Data Type	Memory size	Range
Char	2 bytes	0 to 65535

eg: char ch = 'x';

Boolean Data Type: can handle truth values either true or false

eg:- boolean response = true;

Operators: An operator is a symbol that performs an operation An operator acts on variables called operands

Arithmetic operators: These operators are used to perform fundamental operations like addition, subtraction, multiplication etc

Operator	Meaning	Example	Result
+	Addition	3 + 4	7
-	Subtraction	5 - 7	-2
*	Multiplication	5 * 5	25
/	Division (gives quotient)	14 / 7	2
%	Modulus (gives remainder)	20 % 7	6

• **Assignment operator:** This operator (=) is used to store some value into a variable

Simple Assignment	Compound Assignment
x = x + y	x += y
x = x - y	x -= y
x = x * y	x *= y
x = x / y	x /= y

• **Unary operators:** As the name indicates unary operator's act only on one operand

Operator	Meaning	Example	Explanation
-	Unary minus	j = -k;	k value is negated and stored into j
++	Increment	b++;	b value will be incremented by 1 (called as post incrementation)
	Operator	++b;	b value will be incremented by 1 (called as pre incrementation)
--	Decrement	b--;	b value will be decremented by 1 (called as post decrementation)
	Operator	--b;	b value will be decremented by 1 (called as pre decrementation)

• **Relational operators:** These operators are used for comparison purpose

Operator	Meaning	Example
==	Equal	x == 3
!=	Not equal	x != 3
<	Less than	x < 3
>	Greater than	x > 3
<=	Less than or equal to	x <= 3

• **Logical operators:** Logical operators are used to construct compound conditions A compound condition is a combination of several simple conditions

Operator	Meaning	Example	Explanation
&&	and operator	if(a>b && a>c) System.out.print("yes");	If a value is greater than b and c then only yes is displayed
	or operator	if(a==1 b==1) System.out.print("yes");	If either a value is 1 or b value is 1 then yes is displayed
!	not operator	if(!(a==0)) System.out.print("yes");	If a value is not equal to zero then only yes is displayed

Bitwise operators: These operators act on individual bits (0 and 1) of the operands. They act only on integer data types, i.e. byte, short, long and int.

Operator	Meaning	Explanation
&	Bitwise AND	Multiplies the individual bits of operands
	Bitwise OR	Adds the individual bits of operands
^	Bitwise XOR	Performs Exclusive OR operation
<<	Left shift	Shifts the bits of the number towards left a specified number of positions
>>	Right shift	Shifts the bits of the number towards right a specified number of positions and also preserves the sign bit
>>>	Zero fill right shift	Shifts the bits of the number towards right a specified number of positions and it stores 0 (Zero) in the sign bit
~	Bitwise complement	Gives the complement form of a given number by changing 0's as 1's and vice versa

Ternary Operator or Conditional Operator (? :): This operator is called ternary because it acts on 3 variables. The syntax for this operator is:

Variable = Expression1? Expression2: Expression3;

First Expression1 is evaluated. If it is true, then Expression2 value is stored into variable; otherwise Expression3 value is stored into the variable.

eg: `max = (a > b) ? a : b;`

Program 1: Write a program to perform arithmetic operations.

// Addition of two numbers

class AddTwoNumbers

```
{
    public static void main(String args[])
    {
        int i=10, j=20;
        System.out.println("Addition of two numbers is : " + (i+j));
        System.out.println("Subtraction of two numbers is : " + (i-j));
        System.out.println("Multiplication of two numbers is : " + (i*j));
        System.out.println("Quotient after division is : " + (i/j));
        System.out.println("Remainder after division is : " + (i%j));
    }
}
```

Output:

```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac AddTwoNumbers.java
D:\JQR>java AddTwoNumbers
Addition of two numbers is : 30
Subtraction of two numbers is : -10
Multiplication of two numbers is : 200
Quotient after division is : 0
Remainder after division is : 10
D:\JQR>
```

Program 2: Write a program to perform Bitwise operations

//Bitwise Operations

class Bits

{public static void main(String args[])

{byte x,y;

x=10;

y=11;

System.out.println (~x+"(~x));

System.out.println ("x & y="+x&y));

System.out.println ("x | y="+x|y));

System.out.println ("x ^ y="+x^y));

System.out.println ("x<<2="+x<<2));

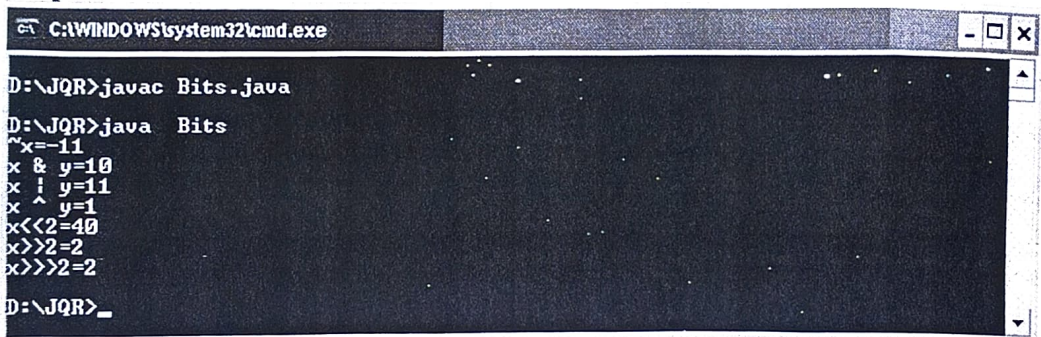
System.out.println ("x>>2="+x>>2));

System.out.println ("x>>>2="+x>>>2));

}

}

Output:



```
C:\WINDOWS\system32\cmd.exe

D:\JQR>javac Bits.java

D:\JQR>java Bits
~x=-11
x & y=10
x | y=11
x ^ y=1
x<<2=40
x>>2=2
x>>>2=2
D:\JQR>_
```


Control Statements

Control statements are the statements which alter the flow of execution and provide better control to the programmer on the flow of execution. In Java, control statements are categorized into selection control statements, iteration control statements and jump control statements.

Java's Selection Statements: Java supports two selection statements: if and switch. These statements allow us to control the flow of program execution based on condition.

if Statement: if statement performs a task depending on whether a condition is true or false.

Syntax: if (condition)

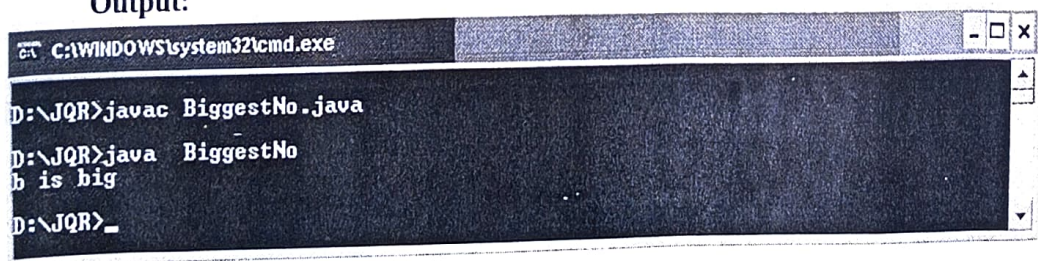
```
        statement1;
    else
        statement2;
```

Here, each statement may be a single statement or a compound statement enclosed in curly braces (that is, a block). The condition is any expression that returns a boolean value. The else clause is optional.

Program 1: Write a program to find the biggest of three numbers.

```
//Biggest of three numbers
class BiggestNo
{
    public static void main(String args[])
    {
        int a=5,b=7,c=6;
        if ( a > b && a > c)
            System.out.println("a is big");
        else if ( b > c)
            System.out.println("b is big");
        else
            System.out.println("c is big");
    }
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac BiggestNo.java
D:\JQR>java BiggestNo
b is big
D:\JQR>
```

Switch Statement: When there are several options and we have to choose only one option from the available ones, we can use the switch statement.

Syntax: switch (expression)

```
{
    case value1: //statement sequence
        break;
    case value2: //statement sequence
```

```

        break;
        .....
        case valueN: //statement sequence
            break;
        default:    //default statement sequence
    }

```

Here, depending on the value of the expression, a particular corresponding case will be executed

Program 2: Write a program for using the switch statement to execute a particular task depending on color value

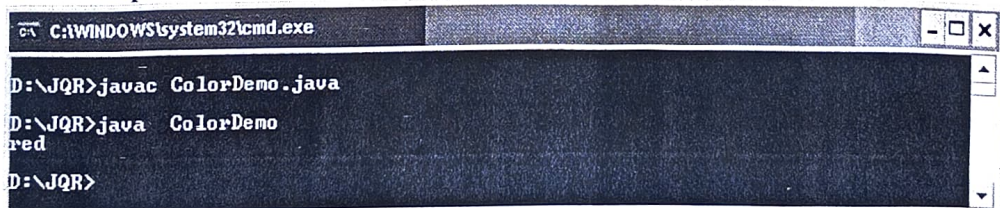
//To display a color name depending on color value class ColorDemo

```

{public static void main(String args[])
{
    char color = „r“;
    switch (color)
    {
        case „r“: System.out.println (“red”);          break;
        case „g“: System.out.println (“green”); break; case
        „b“: System.out.println (“blue”);          break;
        case „y“: System.out.println (“yellow”); break;
        case „w“: System.out.println (“white”); break;
        default: System.out.println (“No Color Selected”);
    }
}
}
}

```

Output:



```

C:\WINDOWS\system32\cmd.exe
D:\JQR>javac ColorDemo.java
D:\JQR>java ColorDemo
red
D:\JQR>

```

Java's Iteration Statements: Java's iteration statements are for, while and do-while. These statements are used to repeat same set of instructions specified number of times called loops. A loop repeatedly executes the same set of instructions until a termination condition is met.

while Loop: while loop repeats a group of statements as long as condition is true. Once the condition is false, the loop is terminated. In while loop, the condition is tested first; if it is true, then only the statements are executed. While loop is called as entry control loop.

Syntax: while (condition)

```

{
    statements;
}

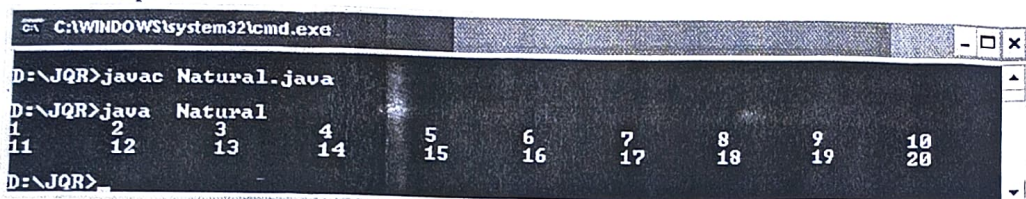
```


Program 3: Write a program to generate numbers from 1 to 20

//Program to generate numbers from 1 to 20

```
class Natural
{
    public static void main(String args[])
    {
        int i=1; while(i <= 20)
        {
            System.out.print(i + "\t"); i++;
        }
    }
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac Natural.java
D:\JQR>java Natural
1      2      3      4      5      6      7      8      9      10
11     12     13     14     15     16     17     18     19     20
D:\JQR>
```

do...while Loop: do...while loop repeats a group of statements as long as condition is true. In do...while loop, the statements are executed first and then the condition is tested. do...while loop is also called as exit control loop.

Syntax:do

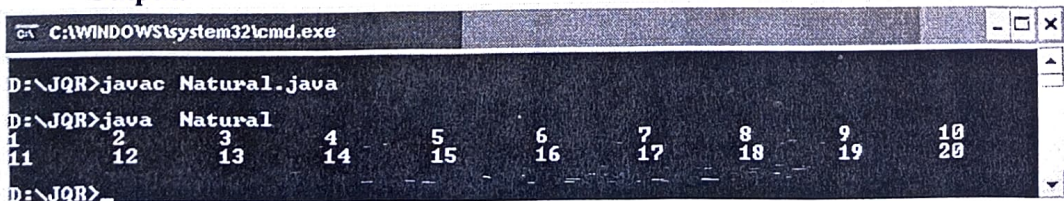
```
{
    statements;
} while (condition);
```

Program 4: Write a program to generate numbers from 1 to 20

//Program to generate numbers from 1 to 20

```
class Natural
{
    public static void main(String args[])
    {
        int i=1; do
        {
            System.out.print(i + "\t"); i++;
        } while (i <= 20);
    }
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac Natural.java
D:\JQR>java Natural
1      2      3      4      5      6      7      8      9      10
11     12     13     14     15     16     17     18     19     20
D:\JQR>
```

for Loop: The for loop is also same as do...while or while loop, but it is more compact syntactically. The for loop executes a group of statements as long as a condition is true.

Syntax:

```
for (expression1; expression2; expression3)
{
    statements;
}
```

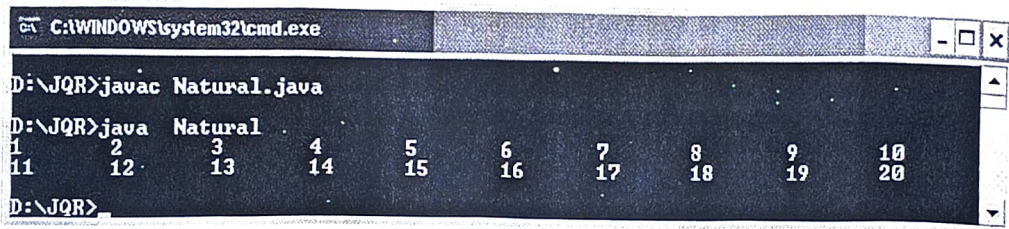
Here, expression1 is used to initialize the variables, expression2 is used for condition checking and expression3 is used for increment or decrement variable value.

Program 5: Write a program to generate numbers from 1 to 20

//Program to generate numbers from 1 to 20

```
class Natural
{
    public static void main(String args[])
    {
        int i;
        for (i=1; i<=20; i++)
            System.out.print (i + "\t");
    }
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac Natural.java
D:\JQR>java Natural
1      2      3      4      5      6      7      8      9      10
11     12     13     14     15     16     17     18     19     20
D:\JQR>
```

Java's Jump Statements: Java supports three jump statements: break, continue and return. These statements transfer control to another part of the program.

o **break:**

break can be used inside a loop to come out of it.

break can be used inside the switch block to come out of the switch block.

break can be used in nested blocks to go to the end of a block. Nested blocks represent a block written within another block.

Syntax: break; (or) break label; //here label represents the name of the block.

Program 6: Write a program to use break as a civilized form of goto

//using break as a civilized form of goto

```
class BreakDemo
{
    public static void main (String args[])
    {
        boolean t = true; first:
        {
            second:
            {
                third:
                {
```


Output:



```
C:\WINDOWS\system32\cmd.exe

D:\JQR>javac BreakDemo.java

D:\JQR>java BreakDemo
Before the break
This is after second block

D:\JQR>
```

continue: This statement is useful to continue the next repetition of a loop/ iteration
When continue is executed, subsequent statements inside the loop are not executed
Syntax: continue;

Program 7: Write a program to generate numbers from 1 to 20

```
//Program to generate numbers from 1 to 20
```

```
class Natural
```

```
{public static void main (String args[])
```

```
{int i=1; while
```

(true)

```
{System.outprint (i + "\t"); i++;
```

```

if (i <= 20 )

```

continue

•

```
else break;
```

}

}

}

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac Natural.java
D:\JQR>java Natural
1      2      3      4      5      6      7      8      9      10
11     12     13     14     15     16     17     18     19     20
D:\JQR>
```

return statement:

return statement is useful to terminate a method and come back to the calling method

return statement in main method terminates the application

return statement can be used to return some value from a method to a calling method

Syntax: return;

(or)

return value; // value may be of any type

Program 8: Write a program to demonstrate return statement

//Demonstrate return

class ReturnDemo

{public static void main(String args[])

{boolean t = true; System.out.println ("Before
the return"); if (t)

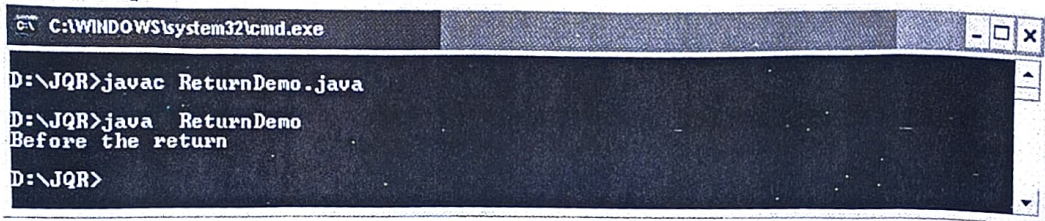
return;

System.out.println ("This won't execute");

}

}

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac ReturnDemo.java
D:\JQR>java ReturnDemo
Before the return
D:\JQR>
```

Note: goto statement is not available in java, because it leads to confusion and forms infinite loops.

Accepting Input from Keyboard

A stream represents flow of data from one place to other place Streams are of two types in java Input streams which are used to accept or receive data Output streams are used to display or write data Streams are represented as classes in java.io package

System.in: This represents InputStream object, which by default represents standard input device that is keyboard

System.out: This represents PrintStream object, which by default represents standard output device that is monitor

System.err: This field also represents PrintStream object, which by default represents monitor System.out is used to display normal messages and results whereas System.err is used to display error messages

To accept data from the keyboard:

Connect the keyboard to an input stream object Here, we can use InputStreamReader that can read data from the keyboard


```
InputStreamReader obj = new InputStreamReader (System.in);
```

Connect InputStreamReader to BufferedReader, which is another input type of stream. We are using BufferedReader as it has got methods to read data properly, coming from the stream.

```
BufferedReader br = new BufferedReader (obj);
```

The above two steps can be combined and rewritten in a single statement as:

```
BufferedReader br = new BufferedReader (new InputStreamReader (System.in));
```

Now, we can read the data coming from the keyboard using `read ()` and `readLine ()` methods available in the `BufferedReader` class.

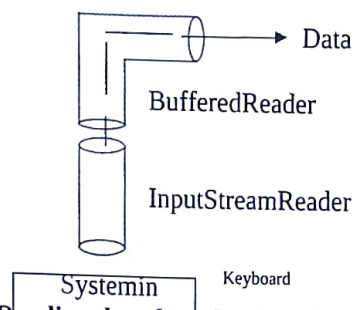


Figure: Reading data from keyboard

Accepting a Single Character from the Keyboard:

Create a `BufferedReader` class object (`br`)

Then read a single character from the keyboard using `read()` method as:

The `read` method reads a single character from the keyboard but it returns its ASCII number, which is an integer. Since, this integer number cannot be stored into character type variable `ch`, we should convert it into `char` type by writing `(char)` before the method. `int` data type is converted into `char` data type, converting one data type into another data type is called type casting.

Accepting a String from Keyboard:

Create a `BufferedReader` class object (`br`)

Then read a string from the keyboard using `readLine()` method

`readLine ()` method accepts a string from keyboard and returns the string into `str`. In this case, casting is not needed since `readLine ()` is taking a string and returning the same data type.

Accepting an Integer value from Keyboard:

First, we should accept the integer number from the keyboard as a string, using `readLine ()` as:

```
String str = br.readLine ();
```

Now, the number is in `str`, i.e. in form of a string. This should be converted into an `int` by using `parseInt ()` method, method of `Integer` class as:

If needed, the above two statements can be combined and written as:

parseInt () is a static method in Integer class, so it can be called using class name as Integer.parseInt ()

We are not using casting to convert String type into int type The reason is String is a class and int is a fundamental data type Converting a class type into a fundamental data type is not possible by using casting It is possible by using the method Integer.parseInt()

Accepting a Float value from Keyboard:

We can accept a float value from the keyboard with the help of the following statement: float n = Float.parseFloat (br.readLine());

We are accepting a float value in the form of a string using br.readLine () and then passing the string to Float.parseFloat () to convert it into float parseFloat () is a static method in Float class

Accepting a Double value from Keyboard:

We can accept a double value from the keyboard with the help of the following statement: double n = Double.parseDouble (br.readLine());

We are accepting a double value in the form of a string using br.readLine () and then passing the string to Double.parseDouble () to convert it into double parseDouble () is a static method in Double class

Accepting Other Types of Values:

- To accept a byte value: byte n = Byte.parseByte (br.readLine ());
- To accept a short value: short n = Short.parseShort (br.readLine ());
- To accept a long value: long n = Long.parseLong (br.readLine ());
- To accept a boolean value: boolean x = Boolean.parseBoolean (br.readLine ());

If read () / readLine () method could not accept values due to some reason (like insufficient memory or illegal character), then it gives rise to a runtime error which is called by the name IOException, where IO stands for Input/Output and Exception represents runtime error But we do not know how to handle this exception, in Java we can use throws command to throw the exception without handling it by writing:

throws IOException at the side of the method where read ()/ readLine () is used

Program 1: Write a program to accept and display student details

Accepting and displaying student

details import java.io.*;

class StudentDemo

{public static void main(String args[]) throws IOException

{// Create BufferedReader object to accept data

BufferedReader br =new BufferedReader (new InputStreamReader
(System.in)); //Accept student details

System.out.print ("Enter roll number: ");

int rno = Integer.parseInt (br.readLine());

System.out.print ("Enter Gender (M/F): ");

char gender = (char)br.read();

brskip (2);

System.out.print ("Enter Student name: ");

String name = br.readLine ()

System.out.println ("Roll No: " + rno);

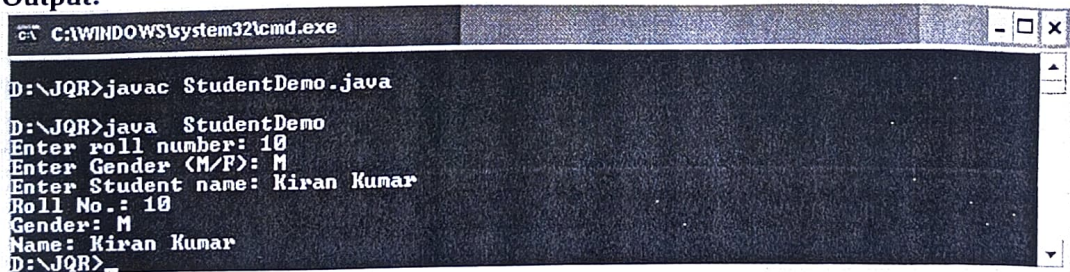
System.out.println ("Gender: " + gender);

System.out.println ("Name: " + name);

}

}

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac StudentDemo.java
D:\JQR>java StudentDemo
Enter roll number: 10
Enter Gender (M/F): M
Enter Student name: Kiran Kumar
Roll No.: 10
Gender: M
Name: Kiran Kumar
D:\JQR>
```

In the above program after accepting gender of the student, brskip (2) is used. The reason is that we used read () method to accept the gender value and then readLine () is used to accept the name. When we type M for gender and press enter, then it releases a \n code. So at gender column, we are giving two characters M and \n. But, read () method takes only the first character and rejects the next character, i.e. \n, which is trapped by the next readLine () method and name will accept \n. For this purpose, we can use skip () method of BufferedReader, which helps in skipping a specified number of characters. Suppose we take \n as two characters; now to skip them, we can write brskip (2);

Arrays and Strings

Arrays: An array represents a group of elements of same data type Arrays are generally categorized into two types:

Single Dimensional arrays (or 1 Dimensional arrays)

Multi-Dimensional arrays (or 2 Dimensional arrays, 3 Dimensional arrays, ...)

Single Dimensional Arrays: A one dimensional array or single dimensional array represents a row or a column of elements For example, the marks obtained by a student in 5 different subjects can be represented by a 1D array

We can declare a one dimensional array and directly store elements at the time of its declaration, as: `int marks[] = {50, 60, 55, 67, 70};`

We can create a 1D array by declaring the array first and then allocate memory for it by using new operator, as: `int marks[];` //declare marks array

`marks = new int[5];` //allot memory for storing 5 elements
These two statements also can be written as: `int marks [] = new int [5];`

Program 1: Write a program to accept elements into an array and display the same

same import javaio*;

class ArrayDemo1

{public static void main (String args[]) throws IOException

{//Create a BufferedReader class object (br)

BufferedReader br = new BufferedReader (new InputStreamReader

(System.in));

System.out.println ("How many elements: ");

int n = Integer.parseInt (br.readLine ());

//create a 1D array with size n

int a[] = new int[n];

System.out.print ("Enter elements into array : ");

for (int i = 0; i < n; i++)

a [i] = Integer.parseInt (br.readLine ());

System.out.print ("The entered elements in the array are:

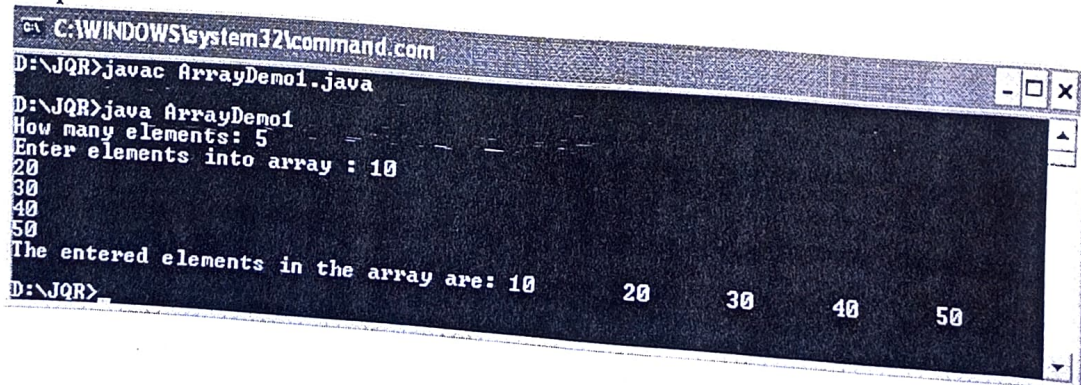
"); for (int i = 0; i < n; i++)

System.out.print (a[i] + "\t");

}

}

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac ArrayDemo1.java
D:\JQR>java ArrayDemo1
How many elements: 5
Enter elements into array : 10
20
30
40
50
The entered elements in the array are: 10 20 30 40 50
D:\JQR>
```


Multi-Dimensional Arrays (2D, 3D ... arrays): Multi dimensional arrays represent 2D, 3D ... arrays A two dimensional array is a combination of two or more (1D) one dimensional arrays A three dimensional array is a combination of two or more (2D) two dimensional arrays

Two Dimensional Arrays (2d array): A two dimensional array represents several rows and columns of data To represent a two dimensional array, we should use two pairs of square braces [] [] after the array name For example, the marks obtained by a group of students in five different subjects can be represented by a 2D array

o We can declare a two dimensional array and directly store elements at the time of its declaration, as:

```
int marks[ ] [ ] = {{50, 60, 55, 67, 70},{62, 65, 70, 70, 81}, {72, 66, 77, 80, 69} };
```

We can create a two dimensional array by declaring the array first and then we can allot memory for it by using new operator as:

```
int marks[ ] [ ]; //declare marks array
```

```
marks = new int[3][5]; //allot memory for storing 15 elements
```

These two statements also can be written as:

```
int marks [ ] [ ] = new int[3][5];
```

Program 2: Write a program to take a 2D array and display its elements in the form of a matrix

//Displaying a 2D array as a matrix

```
class Matrix
```

```
{public static void main(String args[])
```

```
{//take a 2D array
```

```
int x[ ] [ ] = {{1, 2, 3}, {4, 5, 6}}
```

```
}; // display the array elements
```

```
for (int i = 0 ; i < 2 ; i++)
```

```
{System.out.println (); for (int j
```

```
= 0 ; j < 3 ; j++)
```

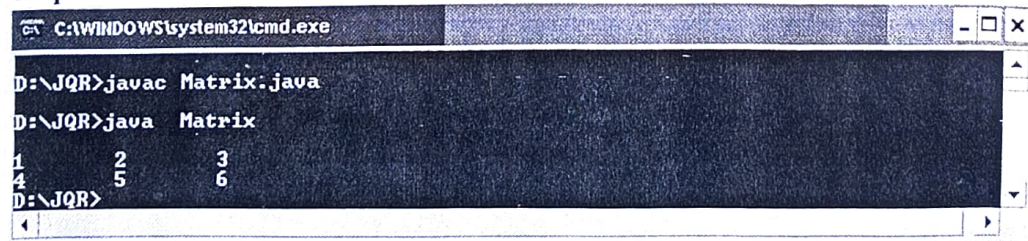
```
System.outprint(x[i][j] + "\t");
```

```
}
```

```
}
```

```
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac Matrix.java
D:\JQR>java Matrix
1 2 3
4 5 6
D:\JQR>
```

Three Dimensional arrays (3D arrays): We can consider a three dimensional array as a combination of several two dimensional arrays To represent a three dimensional array, we should use three pairs of square braces [] [] [] after the array name

o We can declare a three dimensional array and directly store elements at the time of its declaration, as:

```
int arr[ ] [ ] [ ] = {{{50, 51, 52},{60, 61, 62}}, {{70, 71, 72}, {80, 81, 82}}};
```

We can create a three dimensional array by declaring the array first and then we can allot memory for it by using new operator as:

```
int arr[ ][ ] = new int[2][2][3]; //allot memory for storing 15 elements
```

arraynamelength: If we want to know the size of any array, we can use the property „length“ of an array In case of 2D, 3D length property gives the number of rows of the array

Strings: A String represents group of characters Strings are represented as String objects in java

Creating Strings:

We can declare a String variable and directly store a String literal using assignment operator String str = "Hello";

We can create String object using new operator with some data

We can create a String by using character array
also char arr[] = { 'p','r','o','g','r','a','m'};

We can create a String by passing array name to it,
as: String s2 = new String (arr);

We can create a String by passing array name and specifying which characters we need: String s3 = new String (str, 2, 3);

Here starting from 2nd character a total of 3 characters are copied into String s3

String Class Methods:

Method	Description
String concat (String str)	Concatenates calling String with str Note: + also used to do the same
int length ()	Returns length of a String
char charAt (int index)	Returns the character at specified location (from 0)
int compareTo (String str)	Returns a negative value if calling String is less than str, a positive value if calling String is greater than str or 0 if Strings are equal
boolean equals (String str)	Returns true if calling String equals str Note: == operator compares the references of the string objects It does not compare the contents of the objects equals () method compares the contents While comparing the strings, equals () method should be used as it yields the correct result
boolean equalsIgnoreCase (String str)	Same as above but ignores the case
boolean startsWith (String prefix)	Returns true if calling String starts with prefix
boolean endsWith (String suffix)	Returns true if calling String ends with suffix
int indexOf (String str)	Returns first occurrence of str in String
int lastIndexOf (String str)	Returns last occurrence of str in the String Note: Both the above methods return negative value, if str not

	if found in calling String Counting starts from 0
String replace (char oldchar, char newchar)	returns a new String that is obtained by replacing all characters oldchar in String with newchar
String substring (int beginIndex)	returns a new String consisting of all characters from beginIndex until the end of the String
String substring (int beginIndex, int endIndex)	returns a new String consisting of all characters from beginIndex until the endIndex
String toLowerCase ()	converts all characters into lowercase
String toUpperCase ()	converts all characters into uppercase
String trim ()	eliminates all leading and trailing spaces

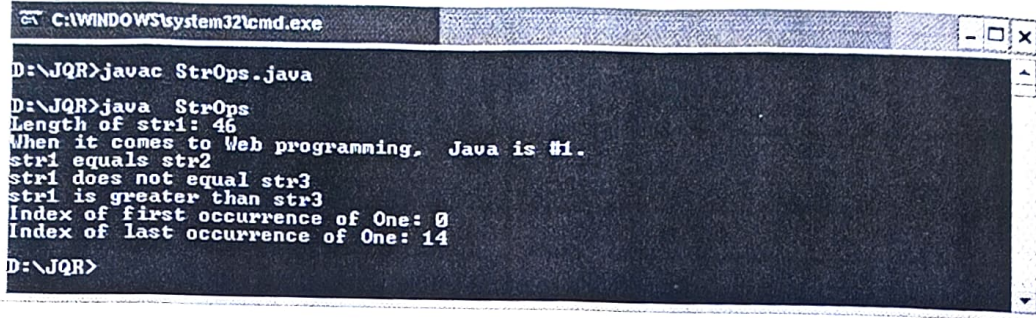
Program 3: Write a program using some important methods of String class
program using String class

methods class StrOps

```
{public static void main(String args [])
{String str1 = "When it comes to Web programming, Java is #1";
String str2 = new String (str1);
String str3 = "Java strings are powerful";
int result, idx; char ch;
System.out.println ("Length of str1: " + str1.length ());
display str1, one char at a
time for(int i=0; i < str1.length();
i++)
System.out.print (str1.charAt (i));
System.out.println ();
if (str1.equals (str2) )
System.out.println ("str1 equals str2");
else
System.out.println ("str1 does not equal
str2"); if (str1.equals (str3) ) System.out.println
("str1 equals str3");
else
System.out.println ("str1 does not equal
str3"); result = str1.compareTo (str3);
if(result == 0)
System.out.println ("str1 and str3 are equal");
else if(result < 0)
System.out.println ("str1 is less than str3");
else
System.out.println ("str1 is greater than str3");
str2 = "One Two Three One"; // assign a new string to str2 idx =
str2.indexOf ("One");
System.out.println ("Index of first occurrence of One: " +
idx); idx = str2.lastIndexOf("One");
System.out.println ("Index of last occurrence of One: " + idx);
}
```

}

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac StrOps.java
D:\JQR>java StrOps
Length of str1: 46
When it comes to Web programming, Java is #1.
str1 equals str2
str1 does not equal str3
str1 is greater than str3
Index of first occurrence of One: 0
Index of last occurrence of One: 14
D:\JQR>
```

We can divide objects broadly as mutable and immutable objects. Mutable objects are those objects whose contents can be modified. Immutable objects are those objects, once created, can not be modified. String objects are immutable. The methods that directly manipulate data of the object are not available in String class.

StringBuffer: StringBuffer objects are mutable, so they can be modified. The methods that directly manipulate data of the object are available in StringBuffer class.

Creating StringBuffer:

We can create a StringBuffer object by using new operator and pass the string to the object, as: `StringBuffer sb = new StringBuffer("Kiran");`

We can create a StringBuffer object by first allotting memory to the StringBuffer object using new operator and later storing the String into it as:

`StringBuffer sb = new StringBuffer(30);`

In general a StringBuffer object will be created with a default capacity of 16 characters. Here, StringBuffer object is created as an empty object with a capacity for storing 30 characters. Even if we declare the capacity as 30, it is possible to store more than 30 characters into StringBuffer. To store characters, we can use append () method as:

`Sbappend("Kiran");`

StringBuffer Class Methods:

Method	Description
StringBuffer append (x)	x may be int, float, double, char, String or StringBuffer. It will be appended to calling StringBuffer.
StringBuffer insert (int offset, x)	x may be int, float, double, char, String or StringBuffer. It will be inserted into the StringBuffer at offset.
StringBuffer delete (int start, int end)	Removes characters from start to end.
StringBuffer reverse ()	Reverses character sequence in the StringBuffer.
String toString ()	Converts StringBuffer into a String.
int length ()	Returns length of the StringBuffer.

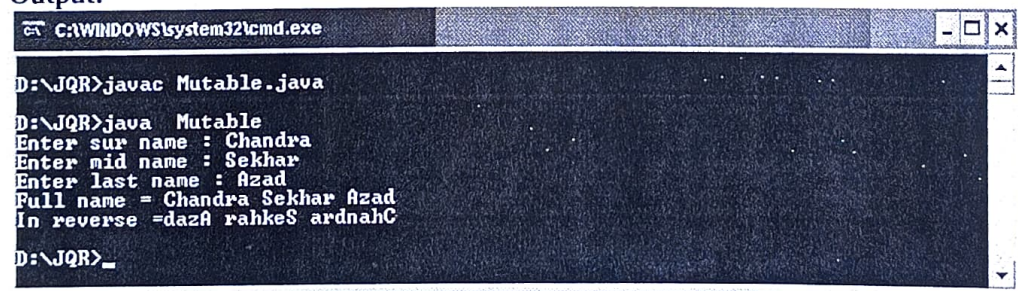
Program 4: Write a program using some important methods of StringBuffer class
program using StringBuffer class

methods import java.io.*;

class Mutable

```
{public static void main(String[] args) throws IOException
    { // to accept data from keyboard
        BufferedReader br=new BufferedReader (new InputStreamReader
            (System.in)); System.out.print ("Enter sur name : ");
        String sur=br.readLine ();
        System.out.print ("Enter mid name : ");
        String mid=br.readLine ();
        System.out.print ("Enter last name : ");
        String last=br.readLine ();
        // create String Buffer object
        StringBuffer sb=new StringBuffer (
            );
        // append sur, last to sb
        sb.append (sur);
        sb.append (last);
        // insert mid after
        // sur int n=sur.length (
        ); sb.insert (n, mid);
        // display full name
        System.out.println ("Full name = "+sb);
        System.out.println ("In reverse =" +sb.reverse ( ));
    }
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac Mutable.java
D:\JQR>java Mutable
Enter sur name : Chandra
Enter mid name : Sekhar
Enter last name : Azad
Full name = Chandra Sekhar Azad
In reverse = dazA rahke$ ardnahC
D:\JQR>
```

Introduction to OOPs

Languages like Pascal, C, FORTRAN, and COBOL are called procedure oriented programming languages. Since in these languages, a programmer uses procedures or functions to perform a task. When the programmer wants to write a program, he will first divide the task into separate sub tasks, each of which is expressed as functions/ procedures. This approach is called procedure oriented approach.

The languages like C++ and Java use classes and object in their programs and are called Object Oriented Programming languages. The main task is divided into several modules and these are represented as classes. Each class can perform some tasks for which several methods are written in a class. This approach is called Object Oriented approach.

Difference between Procedure Oriented Programming and OOP:

Procedure Oriented Programming	Object Oriented Programming
1 Main program is divided into small parts depending on the functions	1 Main program is divided into small object depending on the problem
2 The Different parts of the program connect with each other by parameter passing & using operating system	2 Functions of object linked with object using message passing
3 Every function contains different data	3 Data & functions of each individual object act like a single unit
4 Functions get more importance than data in program	4 Data gets more importance than functions in program
5 Most of the functions use global data	5 Each object controls its own data
6 Same data may be transfer from one function to another	6 Data does not possible transfer from one object to another
7 There is no perfect way for data hiding	7 Data hiding possible in OOP which prevent illegal access of function from outside of it. This is one of the best advantages of OOP also.
8 Functions communicate with other functions maintaining as usual rules	8 One object link with other using the message passing
9 More data or functions can not be added with program if necessary. For this purpose full program need to be change	9 More data or functions can be added with program if necessary. For this purpose full program need not to be change
10 To add new data in program user should be ensure that function allows it	10 Message passing ensure the permission of accessing member of an object from other object
11 Top down process is followed for program design	11 Bottom up process is followed for program design
12 Example: Pascal, Fortran	12 Example: C++, Java

Features of OOP:

Class: In object-oriented programming, a class is a programming language construct that is used as a blueprint to create objects. This blueprint includes attributes and methods that the created objects all share. Usually, a class represents a person, place, or thing - it is an abstraction of a concept within a computer program. Fundamentally, it encapsulates the state.

and behavior of that which it conceptually represents. It encapsulates state through data placeholders called member variables; it encapsulates behavior through reusable code called methods.

General form of a class:

```
class class_name
{
    Properties (variables);
    Actions (methods);
}
```

eg:

```
class Student
{
    //properties -- variables
    int rollNo;
    String name;
    //methods
    void display ()
    {
        System.out.println ("Student Roll Number is: " + rollNo);
        System.out.println ("Student Name is: " + name);
    }
}
```

Note: Variables inside a class are called as instance variables.

Variables inside a method are called as method variables.

Object: An Object is a real time entity. An object is an instance of a class. Instance means physically happening. An object will have some properties and it can perform some actions. Object contains variables and methods. The objects which exhibit similar properties and actions are grouped under one class. "To give a real world analogy, a house is constructed according to a specification. Here, the specification is a blueprint that represents a class, and the constructed house represents the object."

- o To access the properties and methods of a class, we must declare a variable of that class type. This variable does not define an object. Instead, it is simply a variable that can refer to an object.
- o We must acquire an actual, physical copy of the object and assign it to that variable. We can do this using **new** operator. The new operator dynamically allocates memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by new. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated.

General form of an Object:

```
Class_name variable_name; // declare reference to object
variable_name = new Class_name ( ); // allocate an object
```

eg:

```
Student s; // s is reference variable
s = new Student (); // allocate an object to reference variable s
```

The above two steps can be combined and rewritten in a single statement as:

```
Student s = new Student ();
```

Now we can access the properties and methods of a class by using object with dot operator as:

```
s.rollNo, s.name, s.display ()
```

Encapsulation: Wrapping up of data (variables) and methods into single unit is called Encapsulation. A class is an example for encapsulation. Encapsulation can be described as a protective barrier that prevents the code and data from being randomly accessed by other code defined outside the class. Encapsulation is the technique of making the fields in a class private and providing access to the fields via methods. If a field is declared private, it cannot be accessed by anyone outside the class.

eg:

```
class Student
{
    private int rollNo;
    private String name;
    //methods -- actions
    void display ()
    {
        System.out.println ("Student Roll Number is: " + rollNo);
        System.out.println ("Student Name is: " + name);
    }
}
```

Abstraction: Providing the essential features without its inner details is called abstraction (or) hiding internal implementation is called Abstraction. We can enhance the internal implementation without affecting the outside world. Abstraction provides security. A class contains a lot of data and the user does not need the entire data. The advantage of abstraction is that every user will get his own view of the data according to his requirements and will not get confused with unnecessary data. A bank clerk should see the customer details like account number, name, and balance amount in the account. He should not be entitled to see the sensitive data like the staff salaries, profit or loss of the bank etc. So such data can be abstracted from the clerk's view.

eg:

```
class Bank
{
    private int accno;
    private String name;
    private float balance;
    private float profit;
    private float loan;
    void display_to_clerk ()
    {
        System.out.println ("Accno = " + accno);
        System.out.println ("Name = " + name);
        System.out.println ("Balance = " + balance);
    }
}
```

In the preceding class, in spite of several data items, the `display_to_clerk ()` method is able to access and display only the `accno`, `name` and `balance` values. It cannot access `profit`.

and loan of the customer This means the profit and loan data is hidden from the view of the bank clerk

Inheritance: Acquiring the properties from one class to another class is called inheritance (or) producing new class from already existing class is called inheritance Reusability of code is main advantage of inheritance In Java inheritance is achieved by using extends keyword The properties with access specifier private cannot be inherited

```
eg: class Parent
    {
        String parentName;
        String familyName;
    }
    class Child extends Parent
    {
        String childName;
        int childAge;
        void printMyName()
        {
            System.out.println ("My name is"+childName+" "+familyName);
        }
    }
```

In the above example, the child has inherited its family name from the parent class just by inheriting the class

Polymorphism: The word polymorphism came from two Greek words „poly“ means „many“ and „morphos“ means „forms“ Thus, polymorphism represents the ability to assume several different forms The ability to define more than one function with the same name is called Polymorphism

```
eg: int add (int a, int b) float add
    (float a, int b) float
    add (int a , float b)
    void add (float a)
    int add (int a)
```

Message Passing: Calling a method in a class is called message passing We can call methods of a class by using object with dot operator as:

```
object_namemethod_name ();
eg: sdisplay (); obadd (2, 5); obprintMyName ();
```

Program 1: Write a program to display details of student using class and object //Program to display the details of a student using class and object

```
class Student
{
    int rollNo;           //properties -- variables
    String name;
    void display ()       //method -- action
    {
        System.out.println ("Student Roll Number is: " + rollNo);
        System.out.println ("Student Name is: " + name);
    }
}
class StudentDemo
{public static void main(String args[])
```

```

    {

        //create an object to Student class
        Student s = new Student ();
        //call display () method inside Student class using object
        s.display ();
    }
}

```

Output:

```

C:\WINDOWS\system32\cmd.exe
D:\JQR>javac StudentDemo.java
D:\JQR>java StudentDemo
Student Roll Number is: 0
Student Name is: null
D:\JQR>_

```

When the programmer does not initialize the instance variables, java compiler will write code and initializes the variables with default values

Data Type	Default Value
Int	0
Float	00
Double	00
Char	Space
String	null
Class	null
Boolean	false

Initializing Instance Variables:

Type 1: We can initialize instance variables directly in the class using assignment operator
In this type every object is initialized with the same data

```

int rollNo = 101;
String name =
    "Kiran";

```

Program 2: Let us rewrite the Program 1

```

//Program to display the details of a student using class and
object class Student
{int rollNo = 101; String name =
    "Surya"; void display
    ()
    {System.out.println ("Student Roll Number is: " + rollNo);
      System.outprint ("Student Name is: " + name);
    }
}
class StudentDemo
{public static void main(String args[])
    {Student s1 = new Student (); System.outprintln
      ("First Student Details : " );
    }
}

```

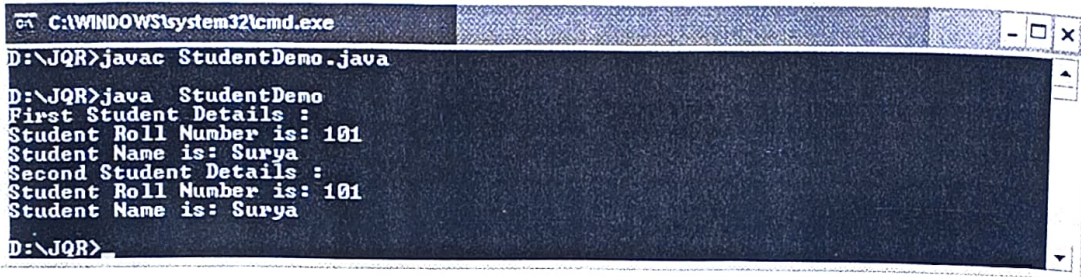


```

        s1display ();
        Student s2 = new Student ();
        System.out.println ("Second Student Details : "
        ); s2display ();
    }
}

```

Output:



```

C:\WINDOWS\system32\cmd.exe
D:\JQR>javac StudentDemo.java
D:\JQR>java StudentDemo
First Student Details :
Student Roll Number is: 101
Student Name is: Surya
Second Student Details :
Student Roll Number is: 101
Student Name is: Surya
D:\JQR>

```

Type 2: We can initialize one class instance variables in another class using reference variable

```

    rollNo = 101;
    sname = "Kiran";

```

Program 3: Let us rewrite the Program 1

//Program to display the details of a student using class and object class Student

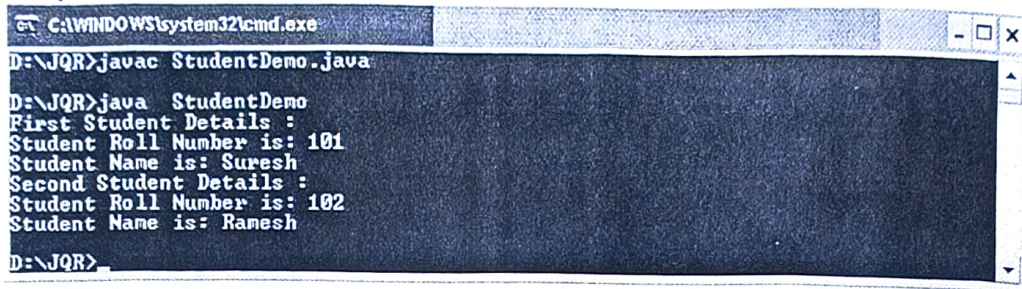
```

{
    int rollNo;
    String name;
    void display ()
    {
        System.out.println ("Student Roll Number is: " + rollNo);
        System.out.print ("Student Name is: " + name);
    }
}

class StudentDemo
{public static void main(String args[])
    {Student s1 = new Student (); System.out.println
        ("First Student Details : "); s1.rollNo = 101;
        s1name = "Suresh";
        s1display ();
        Student s2 = new Student ();
        System.out.println ("Second Student Details : "
        ); s2.rollNo = 102;
        s2name = "Ramesh";
        s2display ();
    }
}

```

.Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac StudentDemo.java
D:\JQR>java StudentDemo
First Student Details :
Student Roll Number is: 101
Student Name is: Suresh
Second Student Details :
Student Roll Number is: 102
Student Name is: Ranesh
D:\JQR>
```

In this type of initialization the properties (variables in the class) are not available, if they are declared as private

Access Specifiers: An access specifier is a key word that represents how to access a member of a class There are four access specifiers in java

private: private members of a class are not available outside the class

public: public members of a class are available anywhere outside the

class o **protected:** protected members are available outside the class

o **default:** if no access specifier is used then default specifier is used by java compiler

Default members are available outside the class

Type 3: We can initialize instance variables using a constructor

Constructor:

o A constructor is similar to a method that initializes the instance variables of a

class o A constructor name and classname must be same

o A constructor may have or may not have parameters Parameters are local variables to receive data

o A constructor without any parameters is called default constructor

eg class Student

```
{int rollNo; String
    name;
    Student ()
    {rollNo = 101; name =
        "Kiran";
    }
}
```

A constructor with one or more parameters is called parameterized constructor

eg class Student

```
{int rollNo; String
    name;

    {rollNo = r;
        name = n;
    }
}
```

A constructor does not return any value, not even void

A constructor is called and executed at the time of creating an object o A constructor is called only once per object

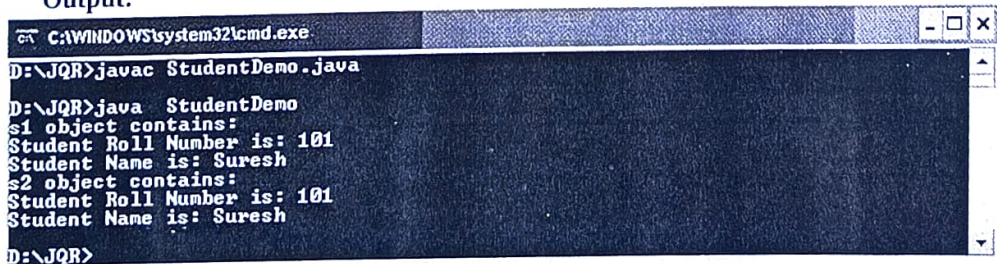
- o Default constructor is used to initialize every object with same data where as parameterized constructor is used to initialize each object with different data
- o If no constructor is written in a class then java compiler will provide default values

Program 4: Write a program to initialize student details using default constructor and display the same

//Program to initialize student details using default constructor and displaying the same class Student

```
{int rollNo; String
    name;
    Student ()
    {rollNo = 101;
        name = "Suresh";
    }
    void display ()
    {System.out.println ("Student Roll Number is: " + rollNo);
        System.out.println ("Student Name is: " + name);
    }
}
class StudentDemo
{public static void main(String args[])
    {Student s1 = new Student (); System.out.println
        ("s1 object contains: "); s1display ();
        Student s2 = new Student ();
        System.out.println ("s2 object contains:
        "); s2display ();
    }
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac StudentDemo.java
D:\JQR>java StudentDemo
s1 object contains:
Student Roll Number is: 101
Student Name is: Suresh
s2 object contains:
Student Roll Number is: 101
Student Name is: Suresh
D:\JQR>
```

Program 5: Write a program to initialize student details using Parameterized constructor and display the same

//Program to initialize student details using parameterized constructor class Student

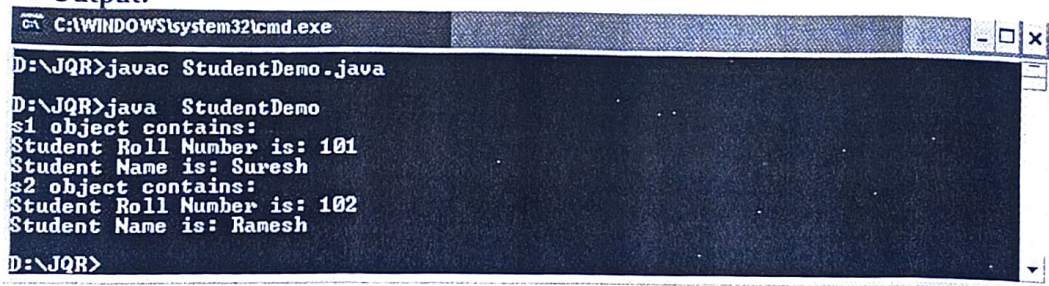
```
{int rollNo;
```

```

String name;
Student (int r, String n)
{rollNo    =    r;
    name = n;
}
void display ()
{System.out.println ("Student Roll Number is: " + rollNo);
    System.out.println ("Student Name is: " + name);
}
}
class StudentDemo
{public static void main(String args[])
    {Student s1 = new Student (101, "Suresh");
        System.out.println ("s1 object contains: ");
        s1display ();
        Student s2 = new Student (102, "Ramesh");
        System.out.println ("s2 object contains: ");
        s2display ();
    }
}

```

Output:



```

C:\WINDOWS\system32\cmd.exe
D:\JQR>javac StudentDemo.java
D:\JQR>java StudentDemo
s1 object contains:
Student Roll Number is: 101
Student Name is: Suresh
s2 object contains:
Student Roll Number is: 102
Student Name is: Ramesh
D:\JQR>

```

The keyword 'this': There will be situations where a method wants to refer to the object which invoked it. To perform this we use „this“ keyword. There are no restrictions to use „this“ keyword. We can use this inside any method for referring the current object. This keyword is always a reference to the object on which the method was invoked. We can use „this“ keyword wherever a reference to an object of the current class type is permitted. „this“ is a key word that refers to present class object. It refers to

- Present class instance variables
- Present class methods
- Present class constructor

Program 6: Write a program to use „this“ to refer the current class parameterized constructor and current class instance variable

```

//this demo
class Person
{String name;

```



```

Person ( )
{this ("Ravi Sekhar"); // calling present class parameterized constructor
  thisdisplay ( ); // calling present class method
}
Person (String name)
{thisname = name; // assigning present class variable with parameter "name"
}
void display ( )
{System.out.println ("Person Name is = " + name);
}
}
class ThisDemo
{public static void main(String args[])
{
    Person p = new Person ( );
}
}

```

Output:



```

C:\WINDOWS\system32\cmd.exe
D:\JQR>javac ThisDemo.java
D:\JQR>java ThisDemo
Person Name is = Ravi Sekhar
D:\JQR>

```

Garbage Collection: Generally memory is allocated to objects by using „new“ operator and deleting an allocated memory is uncommon. This deletion of memory is supported by delete operator in C++ but this deletion of allocated memory works automatically in Java. This automatic deletion of already allocated but unused memory is called as garbage collection. This operation of garbage collection is accomplished by a method named „gc ()“ This method is used for garbage collection.

The finalize () Method: It is possible to define a method that will be called just before an object's final destruction by the garbage collector. This method is called finalize () method. To add a finalizer to a class, simply define the finalize () method. The Java runtime calls that method whenever it is about to recycle an object of that class. Inside the finalize () method specify those actions that must be performed before an object is destroyed. The finalize () method has this general form:

```

protected void finalize ( )
{
    // finalization code here
}

```

Here, the keyword protected is a specifier that prevents access to finalize () by code defined outside its class. This means that you cannot know when or even if finalize () will be executed. For example, if your program ends before garbage collection occurs, finalize () will not execute.

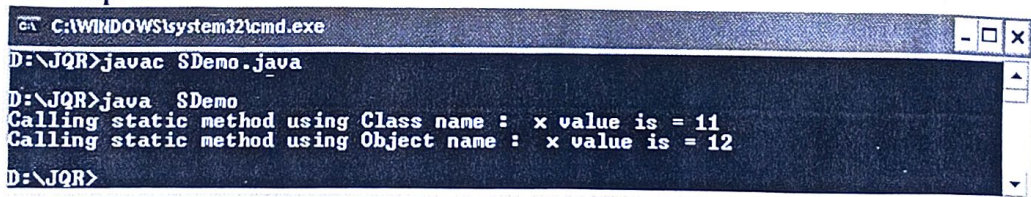
Static Methods:

- o Static methods can read and act upon static variables
- o Static methods cannot read and act upon instance variables
- o Static variable is a variable whose single copy is shared by all the objects
- o Static methods are declared using keyword static
- o Static methods can be called using objectnamemethodname (or) classnamemethodname
- o From any object, if static variable is modified it affects all the objects Static variables are stored on method area

Program 2: Write a program to access static variable using static method //static method accessing static variable

```
class Sample
{static int x = 10; static void
    display( )
    {x++;
        System.out.println (" x value is = " + x);
    }
}
class SDemo
{public static void main(String args[])
    {System.outprint ("Calling static method using Class name : ");
        Sampledisplay ();
        Sample s1 = new Sample ( );
        System.outprint ("Calling static method using Object name :
        "); s1display ();
    }
}
```

Output:



```
C:\WINDOWS\system32\cmd.exe
D:\JQR>javac SDemo.java
D:\JQR>java SDemo
Calling static method using Class name : x value is = 11
Calling static method using Object name : x value is = 12
D:\JQR>
```